Ehsan Kahrizi

Continuous Optimization Using Particle Swarm Optimization

1. Introduction

In this assignment, Particle Swarm Optimization (PSO) is applied to solve a two continuous optimization problems (Eggholder function in two dimensions and the Schwefel function in four dimensions). PSO is a population-based metaheuristic inspired by the social behavior of bird flocking and fish schooling [1]. Each particle in the swarm maintains a position vector (candidate solution), a velocity vector, and memory of its personal best position [2]. The swarm collectively explores the search space by combining individuals cognitive experience with social information sharing [3].

The following components are specified for PSO: (1) termination criterion, (2) swarm size, (3) acceleration coefficients φ_1 and φ_2 controlling cognitive and social influence, (4) neighborhood topology, and optionally (5) inertia weight and/or constriction coefficient for velocity control. Ten distinct PSO configurations is systematically evaluated across five random seeds each, resulting in 50 runs per function and 100 total runs.

2. Problem definition

2.1. The Eggholder Function

The Eggholder function is a two-dimensional optimization benchmark defined as:

$$f(x) = -(x_2+47)\sin(\sqrt{|x_2+x_1/2+47|}) - x_1\sin(\sqrt{|x_1-(x_2+47)|})$$

The domain is $-512 < x_1, x_2 < +512$, with a known global minimum of f (512, 404.2319) = -959.6407. The function has an extremely rugged landscape with numerous local minima, making it a challenging benchmark for optimization algorithms (Figure 1).

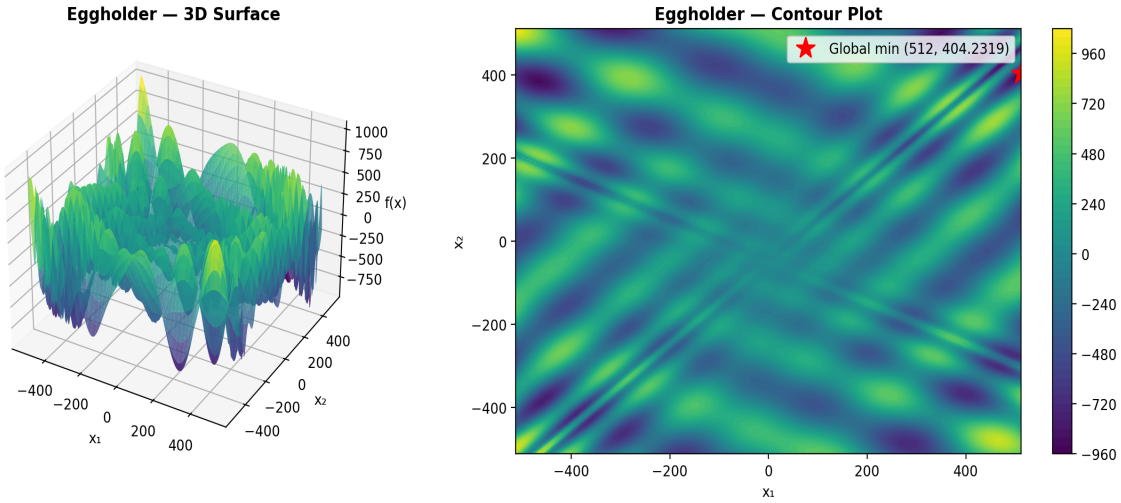

Figure 1 Eggholder function: 3D surface and contour plot.

2.2. The Schwefel Function

The Schwefel function is defined for n dimensions as:

$$f(x_1,\ldots,x_n) = \Sigma_i(-x_i \cdot \sin(\sqrt{|x_i|})) + \alpha \cdot n, \quad \text{where } \alpha = 418.982887$$

For this homework, n = 4 dimensions are used with bounds $-512 \leq x_i \leq 512$. The global minimum is f(420.968746, 420.968746, 420.968746, 420.968746) = 0. The deceptive nature of this function lies in its many local minimal that are far from the global optimum, particularly the large local attractor near x = -512 (Figure 2).

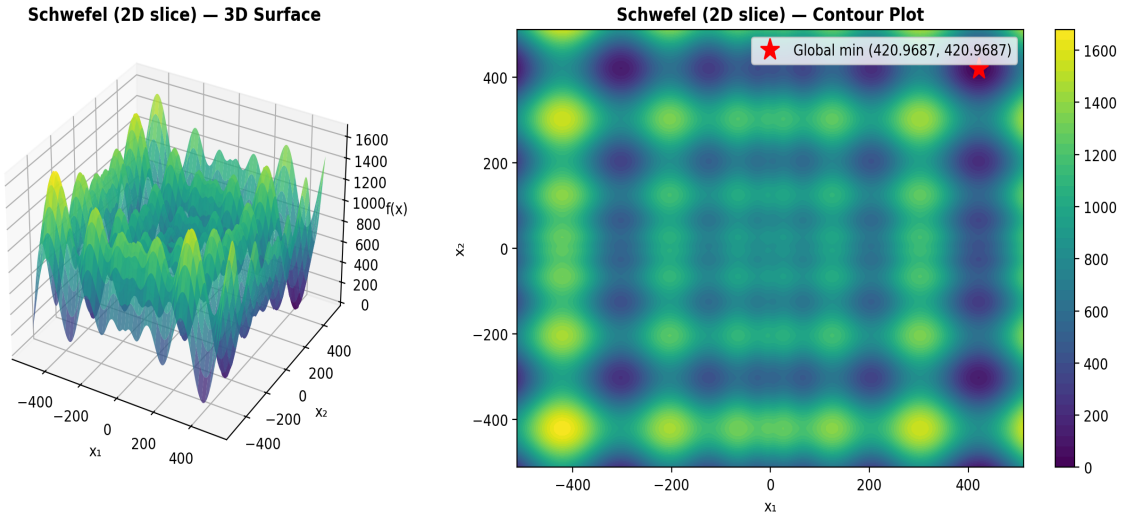

Figure 2 Schwefel function (2D slice): 3D surface and contour plot.

3. Methodology, Definitions

3.1. Solution Encoding

Each particle is represented by a position vector x = (x_1, x_2) for the Eggholder function and x = (x_1, x_2, x_3, x_4) for the Schwefel function. Associated with each particle are a velocity vector v of the same dimensionality, and a personal best position p that stores the best solution found by that particle so far.

3.2. Velocity Update Rule

The standard PSO velocity update for particle i in dimension d is:

$$v_id = v_id + \varphi_1 * r_1 * (p_id - x_id) + \varphi_2 * r_2 * (pgd - x_id)$$

where r_1, r_2 are uniform random numbers in [0,1], p_i is the personal best of particle i, and pg is the best position among particle i's neighbors (determined by the topology). The coefficients φ_1 (cognitive) and φ_2 (social) control the relative influence of individual memory versus social information.

3.3. Parameter Settings

For the base PSO configuration, $\varphi_1 = \varphi_2 = 2.0$ as recommended above. For configurations using constriction, $\varphi_1 = \varphi_2 = 2.05$ (so that $\varphi = 4.1 > 4$, which is required for the constriction coefficient to be well-defined). Swarm sizes are 30 particles for the 2D Eggholder and 50 particles for the

4D Schwefel. All configurations run for 500 iterations. Also, Five random seeds (1, 2, 3, 4, 5) are used consistently across all experiments for both functions.

3.4. Velocity Restriction

Velocities are restricted to [Vmin, Vmax] = [-(ub-lb)/2, (ub-lb)/2] = [-512, 512] for each dimension. If a velocity component exceeds the maximum, it is clamped to the boundary value. This prevents particles from exploding out of the feasible region and improves convergence.

3.5. Boundary Constraint Handling

If updating a particle's position would move it outside the bounds [-512, 512], the position is clamped to the nearest boundary and the velocity in that dimension is reset to zero. This absorbing boundary approach prevents particles from leaving the feasible region while allowing them to explore boundary regions.

3.6. Inertia Weight

When enabled, the inertia weight ω is applied to the previous velocity: $v_id = \omega \cdot v_id + \varphi_1 \cdot r_1 \cdot (p_id - x_id) + \varphi_2 \cdot r_2 \cdot (pgd - x_id)$. The inertia weight decreases linearly from $\omega\_start = 0.9$ to $\omega\_end = 0.4$ over the course of the run. High initial inertia encourages global exploration, while low final inertia promotes local exploitation near promising regions.

3.7. Constriction Coefficient

When enabled, the constriction coefficient K multiplies the entire velocity vector: $v_id = K \cdot [v_id + \varphi_1 \cdot r_1 \cdot (p_id - x_id) + \varphi_2 \cdot r_2 \cdot (pgd - x_id)]$, where $K = 2/|2 - \varphi - \sqrt{(\varphi^2 - 4\varphi)}|$ and $\varphi = \varphi_1 + \varphi_2$. For $\varphi_1 = \varphi_2 = 2.05$, $K \approx 0.7298$. The constriction factor guarantees convergence without requiring explicit velocity clamping, though clamping is still applied as a safety measure.

3.8. Neighborhood Topologies

Two neighborhood topologies are evaluated: (1) Star (Global) topology, where every particle is a neighbor of every other particle, enabling rapid information propagation but susceptibility to premature convergence. (2) Ring topology, where each particle communicates only with its two adjacent neighbors (left and right in a circular arrangement), which slows convergence but preserves diversity and enables more thorough exploration.

3.9. PSO Models

Four models are tested: (1) Full model with both cognitive ($\varphi_1 > 0$) and social ($\varphi_2 > 0$) components. (2) Cognition only with $\varphi_2 = 0$, where particles rely solely on personal memory. (3) Social only with $\varphi_1 = 0$, where particles follow only the neighborhood best. (4) Selfless model with $\varphi_1 = 0$ and the constraint that the global best particle $g \neq i$, meaning the best particle's social component is guided by the second-best particle.

4. Results and Discussion

4.1. Full model

The Table 1 and Figure 3 shows the results for the full model PSO with star (global) topology. It shows strong performance on the Eggholder function. In this setup, all particles share information, which helps the swarm converge quickly. As seen in the results, some runs reached values very close to the global optimum (-959.6407), while others got stuck in local minima

around -894. This leads to some variability across the five seeds, reflected in the standard deviation. Overall, the convergence plot shows that the algorithm improves rapidly in early iterations and then stabilizes, demonstrating efficient but sometimes inconsistent performance due to premature convergence.

Table 1 Eggholder PSO results for full model

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6407	512.0000	404.2326	0.156
2	-894.1509	-463.9381	384.9203	0.142
3	-959.6407	512.0000	404.2317	0.143
4	-959.6404	512.0000	404.2180	0.142
5	-894.4657	-465.2984	386.1655	0.143
Mean	**-933.5077**	**Std: 32.0063**	**Best: -959.6407**	**Worst: -894.1509**

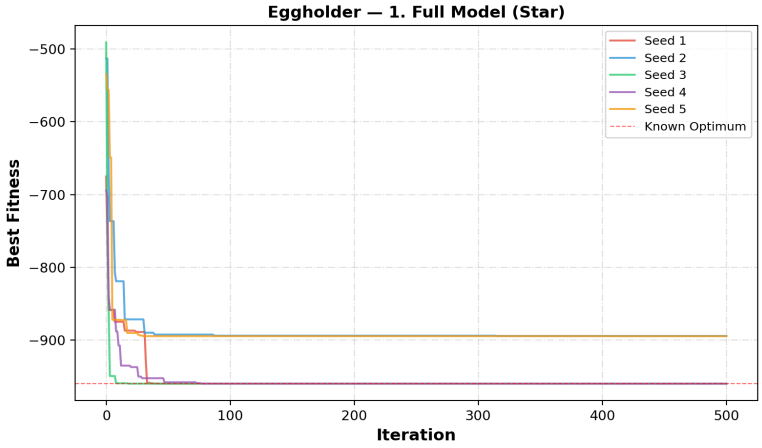

Figure 3 Convergence curves for full model (Star)

4.2. Cognition Only (φ_2=0)

The cognition-only PSO performs very well on the Eggholder function, even without using social information (Figure 4). Since particles rely only on their own experience, they behave like independent hill climbers. The results in the Table 2 show very consistent performance across all five seeds, with all runs finding solutions very close to the global optimum (-959.6407). This is reflected in the very low standard deviation. The convergence curves also show steady improvement, although slightly slower than the full model at the beginning.

Table 2 Eggholder PSO results for cognition only (φ_2=0)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6257	512.0000	404.1171	0.139
2	-959.3798	512.0000	404.7101	0.139
3	-959.6393	512.0000	404.2669	0.140
4	-959.6404	512.0000	404.2462	0.140
5	-959.5960	512.0000	404.4298	0.140
Mean	**-959.5762**	**Std: 0.0995**	**Best: -959.6404**	**Worst: -959.3798**

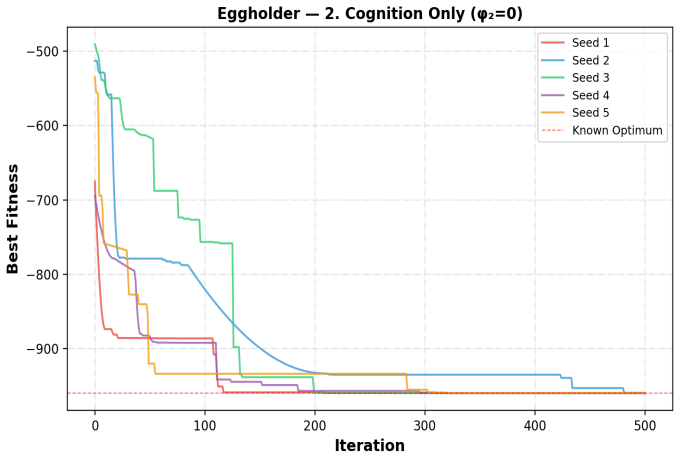

Figure 4 Convergence curves for cognition only ($\varphi_2=0$)

4.3. Social Only ($\varphi_1=0$)

The Table 3 and Figure 5 shows the results for social-only PSO. It performs worse compared to the other models because particles ignore their own experience and follow only the global best. This causes very fast convergence, but often to poor solutions. The results show high variability across the seeds and much worse fitness values compared to the global optimum. This is also reflected in the large standard deviation. From the convergence curves, the algorithm quickly stabilizes but gets stuck in local minima. Overall, this approach lacks exploration.

Table 3 Results for social only ($\varphi_1=0$)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-858.6012	356.3570	512.0000	0.139
2	-786.5182	-456.6559	-382.4537	0.140
3	-858.6009	356.3096	512.0000	0.138
4	-858.6012	356.3450	512.0000	0.138
5	-648.4027	-512.0000	397.5467	0.138
Mean	**-802.1448**	**Std: 81.7836**	**Best: -858.6012**	**Worst: -648.4027**

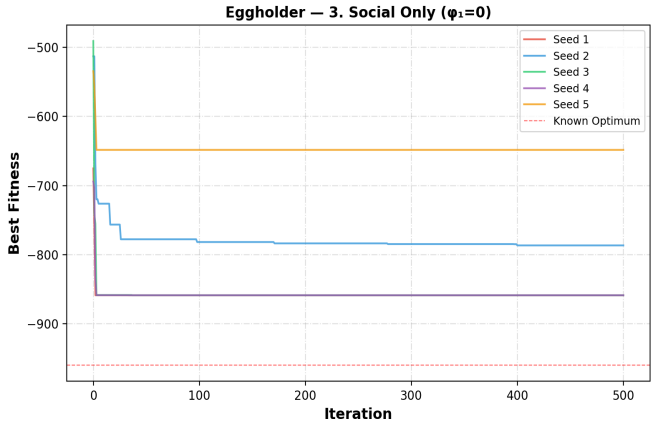

Figure 5 Convergence curves for social only ($\varphi_1=0$)

4.4. Selfless ($\varphi_1=0$, $g\neq i$)

The Table 4 and Figure 6 shows the results for the selfless PSO model. It shows similar behavior to the social-only model, since particles still rely mainly on social information. Although using

the second-best particle helps avoid stagnation of the best particle, the overall performance is still poor. The results show high variability and relatively weak fitness values compared to the global optimum. The convergence curves indicate that the algorithm quickly settles but often in local minima.

Table 4 Results for selfless ($\varphi_1=0$, $g\neq i$)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-858.6012	356.3570	512.0000	0.146
2	-786.3240	-457.9203	-383.7347	0.148
3	-858.6009	356.3106	512.0000	0.144
4	-858.6012	356.3442	512.0000	0.143
5	-648.4027	-512.0000	397.5467	0.143
Mean	**-802.1060**	**Std: 81.7910**	**Best: -858.6012**	**Worst: -648.4027**

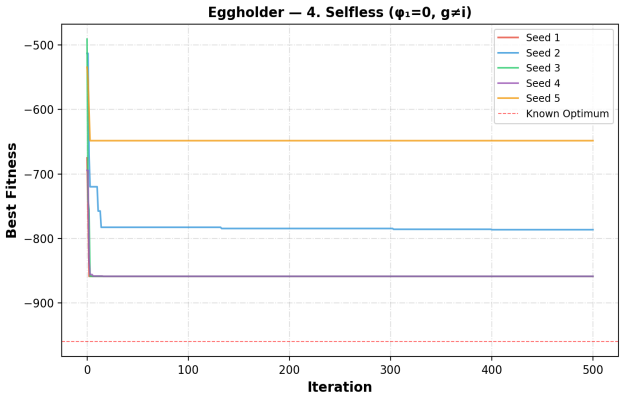

Figure 6 Convergence curves for selfless ($\varphi_1=0$, $g\neq i$)

4.5. Ring Topology (Full)

The ring topology results shows the best overall performance on the Eggholder function. By limiting communication to only two neighbors, the algorithm maintains diversity and avoids premature convergence. The results are very consistent across all seeds, with fitness values very close to the global optimum and a very low standard deviation. The convergence curves show steady improvement and reliable behavior in all runs (Figure 7). Overall, this approach balances exploration and exploitation effectively, making it the most reliable method among the tested models.

Table 5 results for ring topology

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6393	512.0000	404.2661	0.114
2	-959.6396	512.0000	404.2010	0.113
3	-959.6402	512.0000	404.2125	0.115
4	-959.6392	512.0000	404.2674	0.114
5	-959.5233	512.0000	403.9102	0.115
Mean	**-959.6163**	**Std: 0.0465**	**Best: -959.6402**	**Worst: -959.5233**

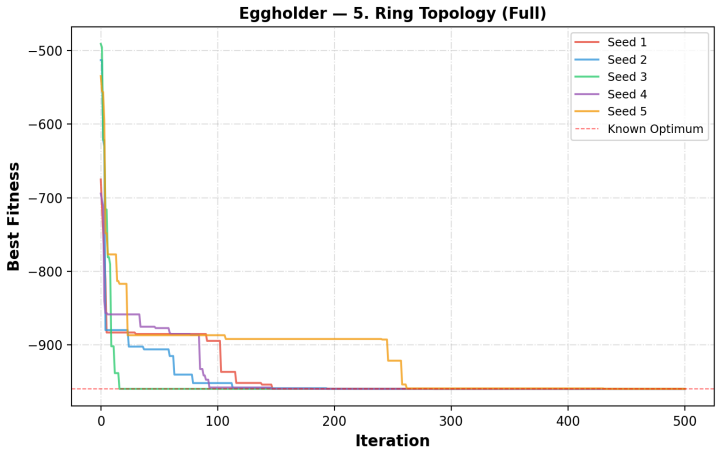

Figure 7 Convergence curves for ring topology (Full)

4.6. Inertia (Star, Full)

The inertia-based PSO improves the search process by balancing exploration and exploitation. At the beginning, a high inertia weight allows particles to explore the search space widely, while later a lower inertia helps fine-tune the solutions. The results show that this model can reach the global optimum in some runs, but not consistently across all seeds. The variation in results is similar to the full model, with some runs getting stuck in local minima. The convergence curves show smoother behavior compared to the basic model.

Table 6 Results for inertia (Star, Full)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6407	512.0000	404.2318	0.138
2	-894.5789	-465.6942	385.7167	0.137
3	-959.6407	512.0000	404.2318	0.138
4	-959.6407	512.0000	404.2318	0.142
5	-894.5789	-465.6942	385.7167	0.137
Mean	**-933.6160**	**Std: 31.8736**	**Best: -959.6407**	**Worst: -894.5789**

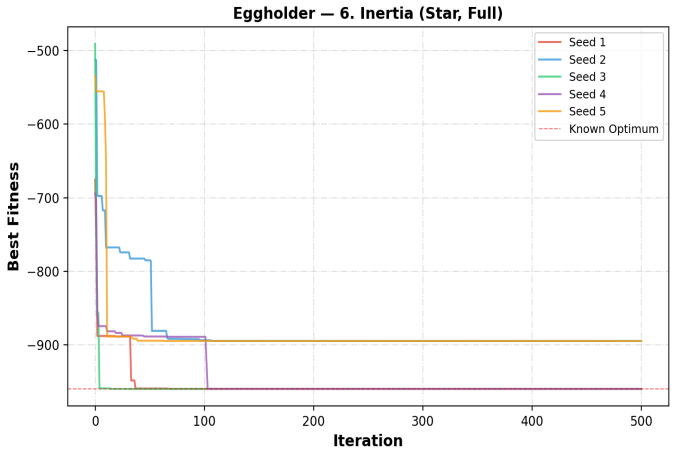

Figure 8 Convergence curves for inertia (Star, Full)

4.7. Constriction (Star, Full)

The constriction-based PSO aims to improve convergence stability by controlling particle velocities. The constriction coefficient $K \approx 0.7298$ is applied with $\varphi_1 = \varphi_2 = 2.05$. While it can

reach the global optimum in some runs, the results are inconsistent across different seeds. Some runs perform well, but others get stuck in local minima, leading to higher variability. This is reflected in the relatively large standard deviation. The convergence curves show that the algorithm stabilizes quickly but does not always find the best solution. Overall, although constriction provides theoretical benefits, it does not consistently improve performance compared to simpler methods.

Table 7 Results for constriction (Star, Full)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6407	512.0000	404.2318	0.143
2	-786.5260	-456.8858	-382.6234	0.142
3	-959.6407	512.0000	404.2318	0.141
4	-888.9491	347.3270	499.4154	0.147
5	-894.5789	-465.6942	385.7167	0.142
Mean	**-897.8671**	**Std: 63.4336**	**Best: -959.6407**	**Worst: -786.5260**

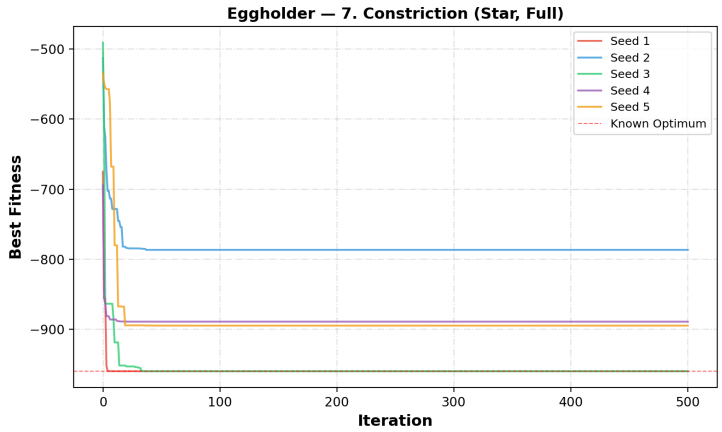

Figure 9 Convergence curves for constriction (Star, Full)

4.8. Inertia + Constriction

The combination of inertia and constriction aims to improve both exploration and stability in PSO. However, the results show mixed performance. While some runs successfully reach the global optimum, others get stuck in poor local minima, leading to high variability. This is reflected in the large standard deviation. The convergence curves show that the algorithm can converge quickly, but not always to the best solution. Overall, combining both methods does not consistently improve performance and can sometimes make the results less stable.

Table 8 Results for inertia + constriction

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6407	512.0000	404.2318	0.142
2	-753.0502	-390.3739	-415.0902	0.142
3	-959.6407	512.0000	404.2318	0.143
4	-959.6407	512.0000	404.2318	0.141
5	-821.1965	-313.9720	512.0000	0.141
Mean	**-890.6337**	**Std: 87.2200**	**Best: -959.6407**	**Worst: -753.0502**

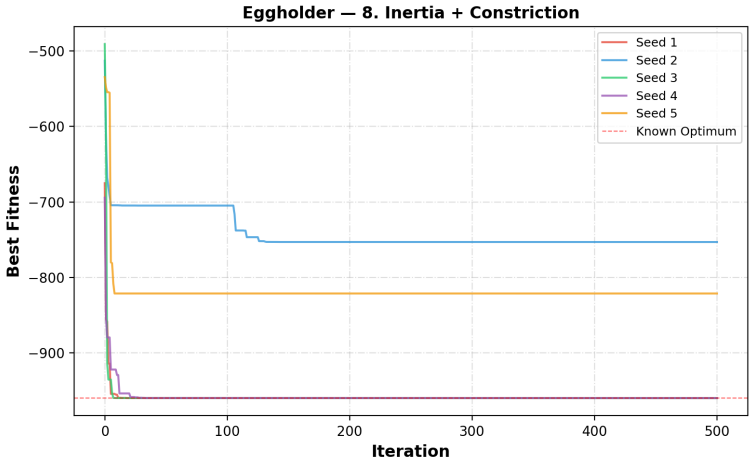

Figure 10 Convergence curves for inertia + constriction

4.9. More Particles (Inertia+Constr)

Increasing the number of particles to 60 improves the exploration ability of the PSO algorithm. With more particles, the swarm can cover a larger portion of the search space, increasing the chances of finding the global optimum. As shown in the results, several runs successfully reached the optimal value (-959.6407). Compared to smaller swarms sizes, the performance is more reliable, although not perfect across all seeds. The mean fitness is better than some previous configurations, indicating an overall improvement.

However, this improvement comes with a higher computational cost, as seen in the increased runtime. Additionally, not all runs perform equally well, and some still get trapped in local minima, leading to noticeable variability. The convergences curves show that while some particles quickly find the optimal region, others converge more slowly or to suboptimal solutions. I think in general, using more particles enhances performance and reliability, but it does not completely eliminate the risk of premature convergence.

Table 9 Results for more particles (Inertia+Constr)

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-959.6407	512.0000	404.2318	0.346
2	-786.5260	-456.8858	-382.6234	0.345
3	-894.5789	-465.6942	385.7167	0.344
4	-959.6407	512.0000	404.2318	0.345
5	-959.6407	512.0000	404.2318	0.346
Mean	**-912.0054**	**Std: 67.6108**	**Best: -959.6407**	**Worst: -786.5260**

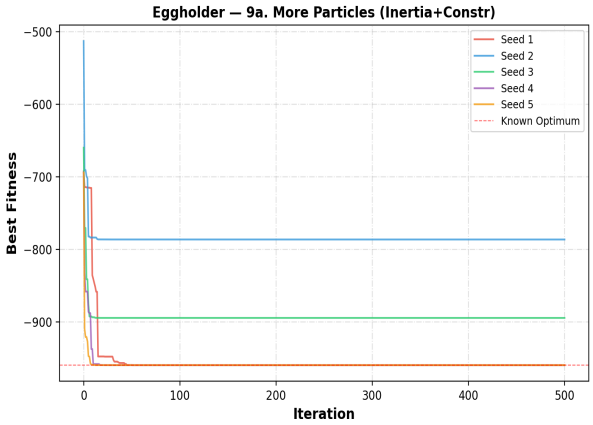

Figure 11 Convergence curves for more particles

4.10. Fewer Particles

Reducing the number of particles to 15 significantly decreases the exploration ability of the PSO algorithm. With fewer particles, the swarm cannot cover the search space effectively, which reduces the chances of finding the global optimum. The results show poor and inconsistent performance across the seeds, with some runs getting far from the optimal value. This is reflected in the low mean fitness and very high standard deviation, indicating large variability in performance.

Although the runtime is much lower due to fewer particles, this comes at the cost of solution quality. The convergence curves show that the algorithm often stabilizes quickly, but in suboptimal regions. Only one run gets close to the global optimum, while others are trapped in local minima. Overall, using fewer particles reduces computational cost but significantly harms performance and reliability.

Table 10 Results for fewer particles

Seed	Best Fitness	Best x_1	Best x_2	Runtime (s)
1	-888.9491	347.3270	499.4154	0.063
2	-894.5789	-465.6942	385.7167	0.063
3	-565.9978	-105.8769	423.1532	0.063
4	-956.9182	482.3533	432.8790	0.063
5	-718.1675	283.0759	-487.1257	0.063
Mean	**-804.9223**	**Std: 143.3966**	**Best: -956.9182**	**Worst: -565.9978**

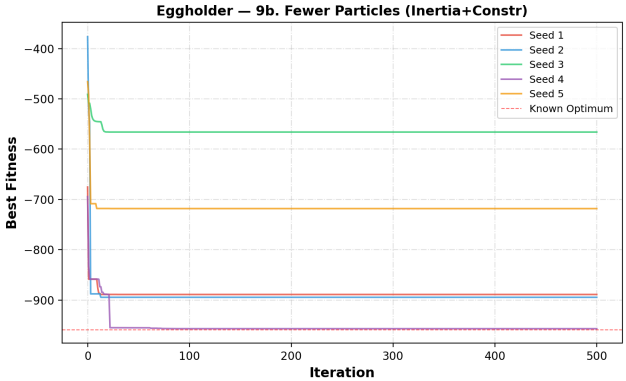

Figure 12 Convergence curves for rewer particles

4.11 Overall Comparison

The overall comparison of all PSO variants on the Eggholder function is summarized Table 11, which presents the mean, standard deviation, best, and worst fitness values for each method. As required in the assignment, each configuration was evaluated using five different seeds to ensure a fair comparison. From the table, the ring topology (full model) achieves the best mean fitness and lowest standard deviation, indicating both high accuracy and strong consistency. The cognition-only model also performs very well with similarly low variability. In contrast, the social-only and selfless models show much worse mean fitness and very high standard deviations, indicating unstable and poor performance.

The performance differences are also clearly visible in Figure 13 and Figure 14**.**These figures show that the ring topology and cognition-only models consistently produce results close to the global optimum with very little spread. On the other hand, the social-only and selfless models have wide distributions, confirming their high variability. The convergence behavior across all methods is illustrated in Figure 16, where it can be seen that the better-performing methods improve gradually and reliably, while weaker methods converge quickly but get stuck in local minima. This highlights the importance of maintaining diversity rather than relying only on fast convergence.

Finally, the impact of swarm size is demonstrated in Figure 15, which compares different particle counts. Increasing the number of particles (as seen in the "more particles" case in Table 11) improves exploration and increases the likelihood of reaching the global optimum, although it also increases runtime. In contrast, using fewer particles leads to poor performance and high variability, as shown by the worst mean fitness and largest standard deviation in Table 11. Overall, the results confirm that the assignment objectives were met by systematically comparing all PSO variants, and they show that methods preserving diversity—especially ring topology and cognition-only—are the most effective for this problem.

Table 11 Summary of all PSO variants for the Eggholder function

Variant	Particles	Mean	Std	Best	Worst
Full Model (Star)	30	-933.5077	32.0063	-959.6407	-894.1509
Cognition Only (φ_2=0)	30	-959.5762	0.0995	-959.6404	-959.3798
Social Only (φ_1=0)	30	-802.1448	81.7836	-858.6012	-648.4027
Selfless (φ_1=0, g≠i)	30	-802.1060	81.7910	-858.6012	-648.4027
Ring Topology (Full)	30	-959.6163	0.0465	-959.6402	-959.5233
Inertia (Star, Full)	30	-933.6160	31.8736	-959.6407	-894.5789
Constriction (Star, Full)	30	-897.8671	63.4336	-959.6407	-786.5260
Inertia + Constriction	30	-890.6337	87.2200	-959.6407	-753.0502
More Particles (Inertia+Constr)	60	-912.0054	67.6108	-959.6407	-786.5260
Fewer Particles (Inertia+Constr)	15	-804.9223	143.3966	-956.9182	-565.9978

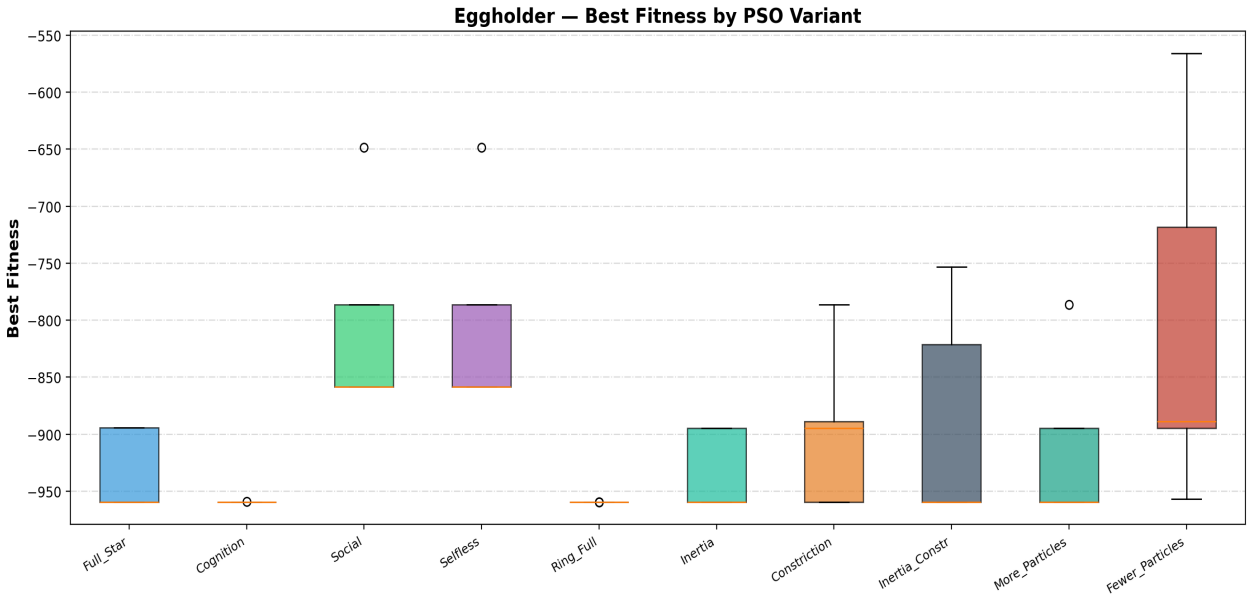

Figure 13 Boxplot comparison of all PSO variants

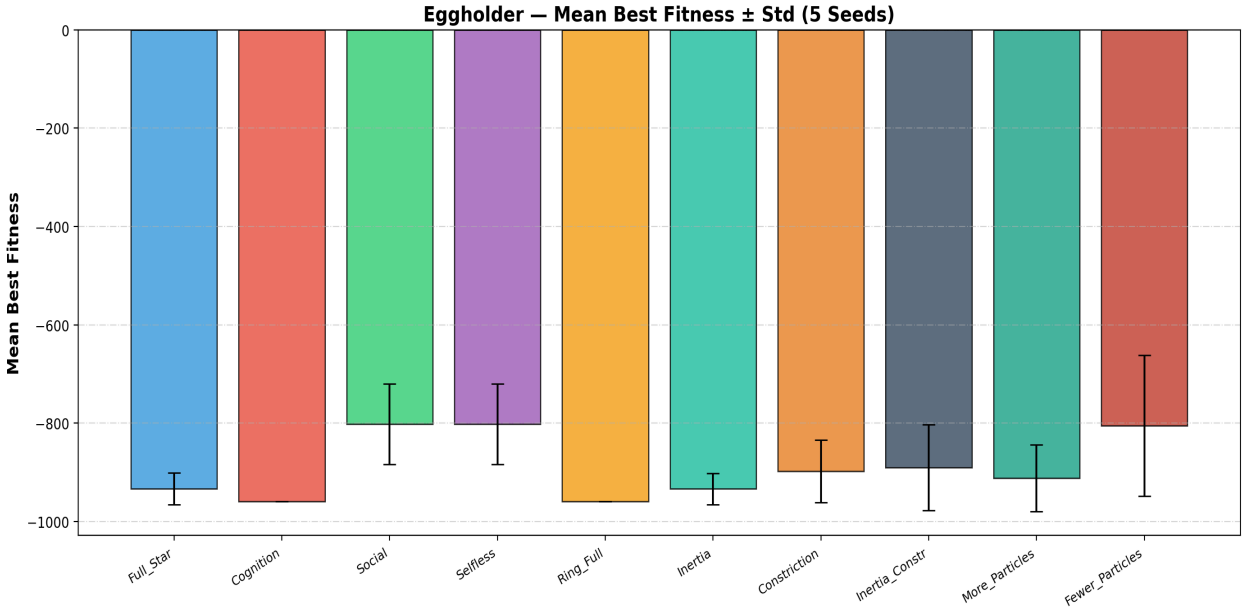

Figure 14 Mean best fitness (± standard deviation) for all PSO variants

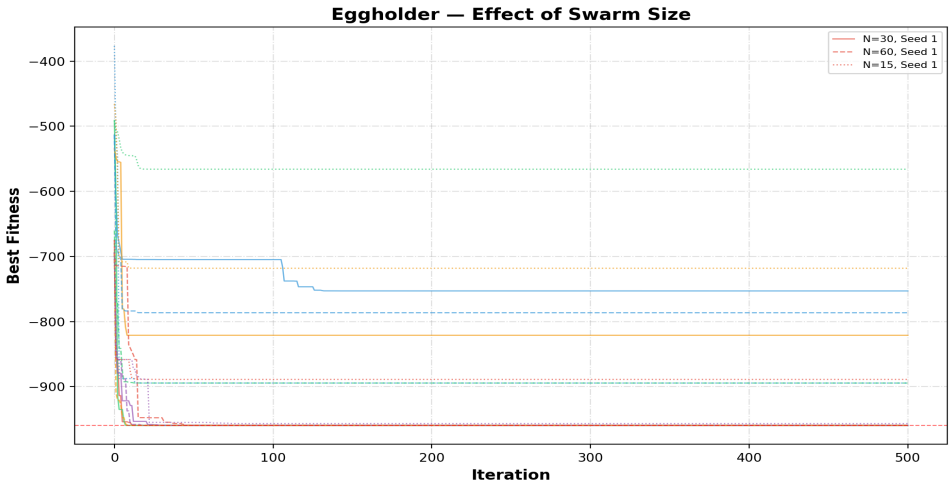

Figure 15 Effect of swarm size on PSO performance

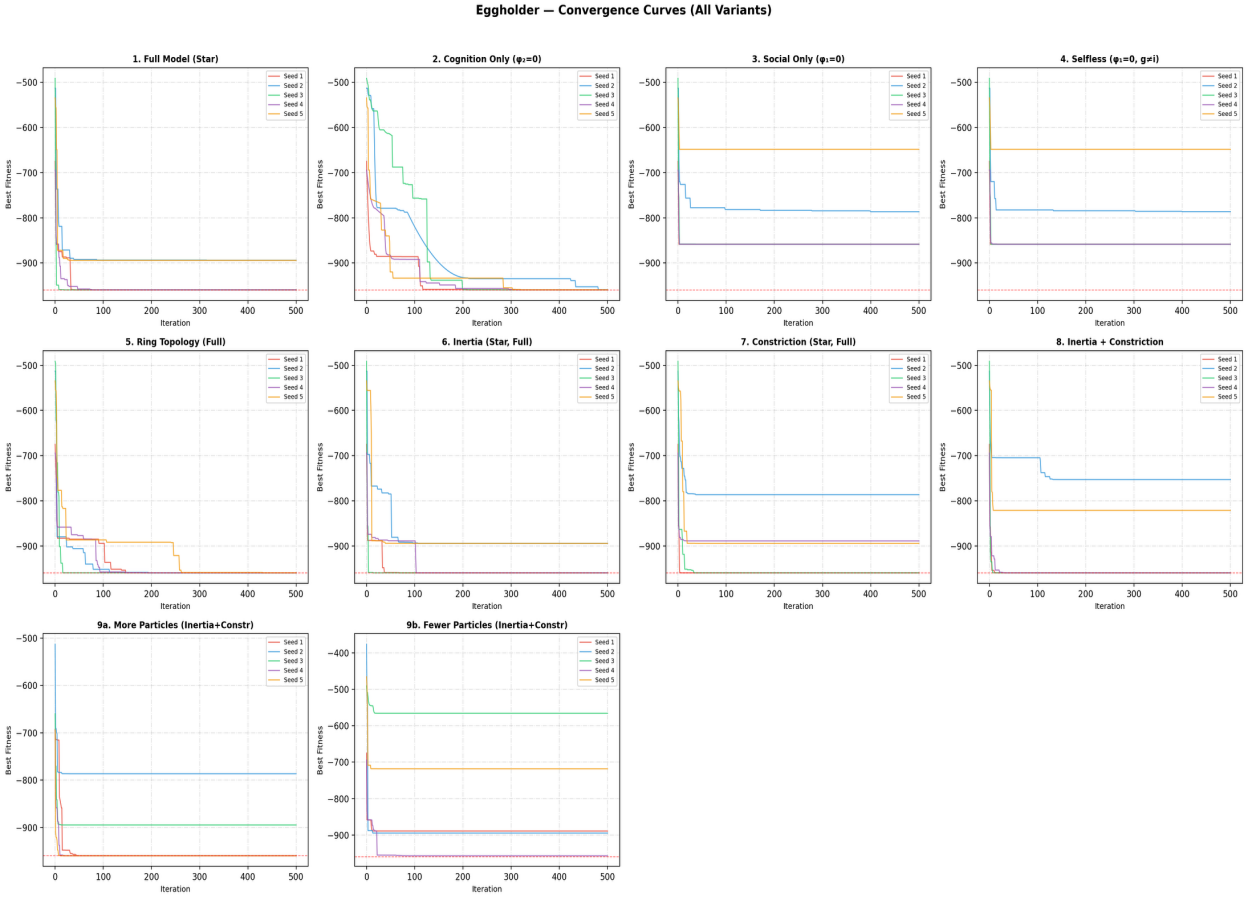

Figure 16 Convergence curves for all PSO variants

5. Bonus

The same previous section variants are applied to the 4-dimensional Schwefel function. The same five random seeds (1, 2, 3, 4, 5) are used without shuffling or randomization to maintain consistency. The default swarm size is increased to 50 particles to account for the higher dimensionality, with 100 and 25 particles for the size-variation experiments.

5.1. Full Model (Star)

For the 4D Schwefel function, the full model with star topology shows more difficulty compared to the Eggholder case due to the higher dimensionality and many deceptive local minima. As shown in Table 12, the results vary significantly across the five seeds, with relatively high mean fitness and noticeable standard deviation. Only one run achieves a much better solution, while others get trapped far from the global optimum. This behavior is also visible in Figure 17, where the convergence curves show rapid initial improvement but early stagnation in suboptimal regions. Overall, the full model struggles to consistently find good solutions for the Schwefel function, highlighting the increased complexity of the problem and the need for better exploration strategies.

Table 12 Schwefel PSO results for full model (Star)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.8032	419.9357, -512.0000, -511.9858, 421.4087		0.329
2	118.4373	419.5024, 420.4125, -512.0000, 426.1378		0.331
3	229.5997	421.3706, 420.2104, -512.0000, -512.0000		0.329
4	229.6854	-512.0000, 421.6138, -511.9960, 421.8012		0.329
5	229.8916	-512.0000, 419.4223, -512.0000, 421.7806		0.329
Mean	**207.4835**	**Std: 44.5232**	**Best: 118.4373**	**Worst: 229.8916**

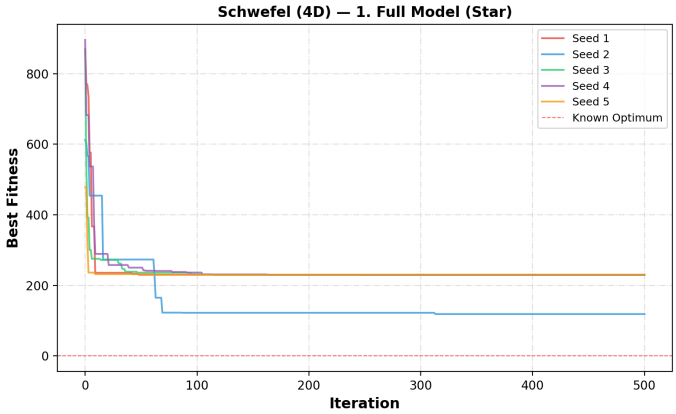

Figure 17 Convergence curves for full model (Star) on the Schwefel function

5.2. Cognition Only (φ_2=0)

The cognition-only PSO performs significantly better on the 4D Schwefel function compared to the full model. As shown in Table 13, the results are more consistent, with a lower mean fitness and smaller standard deviation. Some runs achieve relatively good solutions close to the global optimum, especially the best run with a fitness of 36.6424. From Figure 18, the convergence curves show steady improvement over time, although the algorithm still does not consistently reach the optimal value of 0. Overall, relying on personal experience helps maintain diversity and improves performance, but the problem remains challenging due to its many local minima.

Table 13 Schwefel (4D) PSO results for cognition only ($\varphi_2=0$)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	130.7328	425.7624, 419.8792, 421.6665, -293.9686		0.323
2	109.6883	438.4705, 407.9496, 430.0197, 438.8389		0.324
3	117.7478	423.0890, 421.7994, -512.0000, 416.6570		0.325
4	72.9305	431.9353, 409.2442, 426.8603, 437.9853		0.325
5	36.6424	421.0844, 409.6588, 425.7120, 409.0097		0.328
Mean	**93.5484**	**Std: 34.3284**	**Best: 36.6424**	**Worst: 130.7328**

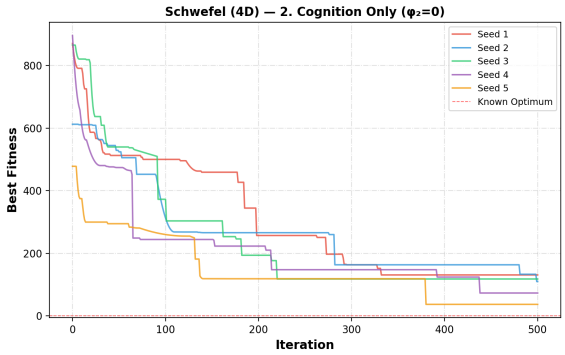

Figure 18 Convergence curves for cognition only ($\varphi_2=0$) on the Schwefel function

5.3. Social Only ($\varphi_1=0$)

The social-only PSO performs poorly on the 4D Schwefel function, as shown in Table 14. The results have a high mean fitness and very large standard deviation, indicating unstable and inconsistent performance across different seeds. Although one run achieves a relatively good solution (fitness ≈ 11.49), most runs are trapped far from the global optimum, often near the boundary at -512. This behavior is also seen in Figure 19, where the convergence curves show very fast initial improvement followed by early stagnation. Overall, relying only on social information leads to premature convergence and weak exploration, making this approach ineffective for this complex problem.

Table 14 Schwefel PSO results for social only ($\varphi_1=0$)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.5511	420.8171, 420.3958, -512.0000, -512.0000		0.324
2	11.4895	418.7611, 419.0848, 427.4217, 427.3692		0.325
3	229.5071	420.9340, 420.9376, -512.0000, -512.0000		0.324
4	229.6018	-512.0000, 420.4023, -512.0000, 420.3113		0.327
5	229.7059	-512.0000, 419.7412, -512.0000, 420.7011		0.328
Mean	**185.9711**	**Std: 87.2408**	**Best: 11.4895**	**Worst: 229.7059**

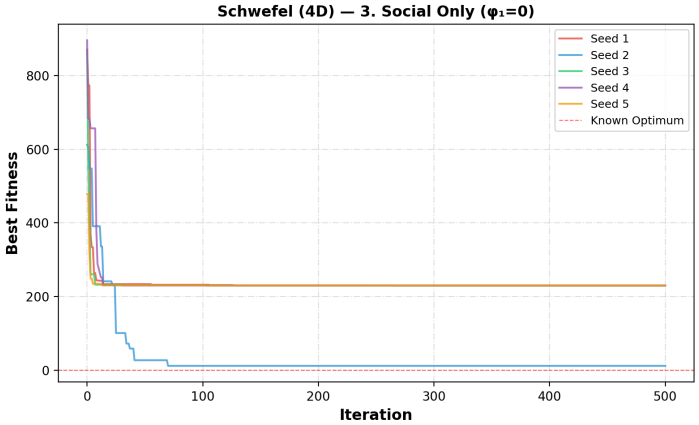

Figure 19 Convergence curves for social only ($\varphi_1=0$) on the Schwefel function

5.4. Selfless ($\varphi_1=0$, $g\neq i$)

The selfless PSO model shows similar performance to the social-only approach on the 4D Schwefel function. As seen in Table 15, the results have a high mean fitness and moderate standard deviation, indicating that the algorithm is not able to consistently find good solutions. Although using the second-best particle helps avoid stagnation of the best particle, it does not significantly improve overall performance. From Figure 20, the convergence curves show fast initial progress followed by early stagnation in suboptimal regions. Overall, this model still lacks sufficient exploration and struggles with the deceptive nature of the Schwefel function.

Table 15 Schwefel (4D) PSO results for selfless ($\varphi_1=0$, $g\neq i$)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.5538	421.1250, 420.3784, -512.0000, -512.0000		0.347
2	114.9455	421.0705, -512.0000, 422.1847, 421.1492		0.346
3	229.5071	420.9340, 420.9376, -512.0000, -512.0000		0.342
4	229.5487	-512.0000, 421.4689, -512.0000, 421.2547		0.341
5	229.7033	-512.0000, 420.3249, -512.0000, 422.0375		0.343
Mean	**206.6517**	**Std: 45.8532**	**Best: 114.9455**	**Worst: 229.7033**

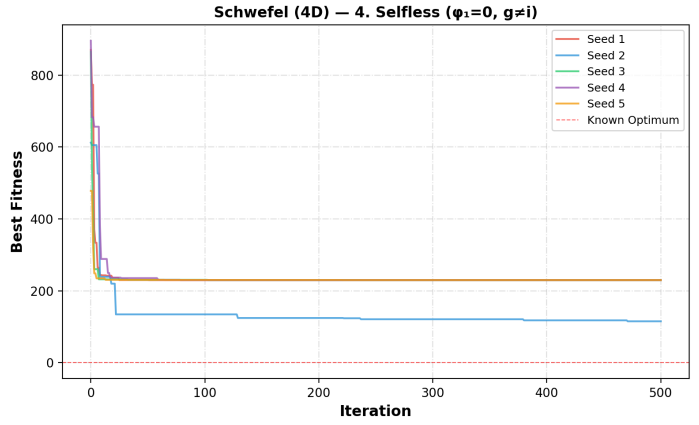

Figure 20 Convergence curves for selfless ($\varphi_1=0$, $g\neq i$) on the Schwefel function (4D)

5.5. Ring Topology (Full)

The ring topology limits each particle to two neighbors (left and right in a circular list). This slows information propagation, enabling better exploration of the search space.

Table 16 Schwefel (4D) PSO results for ring topology (Full)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	122.0768	428.2484, 421.0646, -512.0000, 418.7253		0.249
2	68.1561	413.6649, 407.3081, 403.7071, 417.5449		0.250
3	41.3782	433.5632, 409.2268, 419.4420, 415.3757		0.247
4	71.0992	404.9261, 405.9628, 422.1091, 430.3746		0.249
5	93.8713	408.3414, 436.0713, 419.1128, 401.8455		0.256
Mean	**79.3163**	**Std: 27.0986**	**Best: 41.3782**	**Worst: 122.0768**

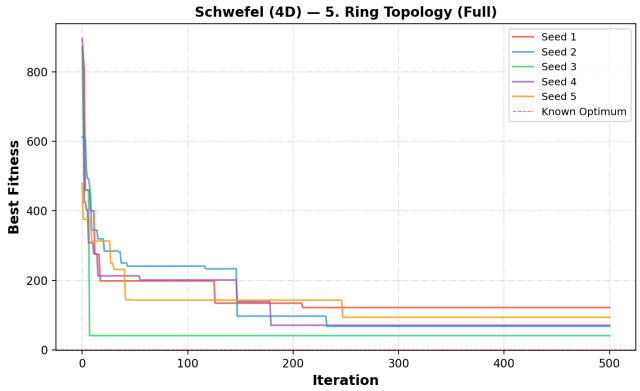

Figure 21 Convergence curves for ring topology (Full) on the Schwefel function (4D)

5.6. Inertia (Star, Full)

The inertia-based PSO shows improved performance on the 4D Schwefel function compared to earlier models. As shown in Table 17, one of the runs successfully reaches the global optimum (fitness ≈ 0), demonstrating the benefit of balancing exploration and exploitation. However, the results are still inconsistent, with other runs getting trapped in local minima, leading to a high standard deviation. From Figure 22, the convergence curves show that some particles are able to escape poor regions and continue improving, while others stagnate early. Overall, inertia helps the algorithm explore more effectively, but it does not guarantee consistent convergence to the global optimum.

Table 17 Schwefel PSO results for inertia (Star, Full)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.5068	420.9687, 420.9687, -512.0000, -512.0000		0.317
2	-0.0000	420.9687, 420.9687, 420.9687, 420.9687		0.319
3	229.5068	420.9687, 420.9687, -512.0000, -512.0000		0.317
4	114.7534	420.9687, 420.9687, -512.0000, 420.9687		0.318
5	229.5068	-512.0000, 420.9687, -512.0000, 420.9687		0.316
Mean	**160.6548**	**Std: 91.8027**	**Best: -0.0000**	**Worst: 229.5068**

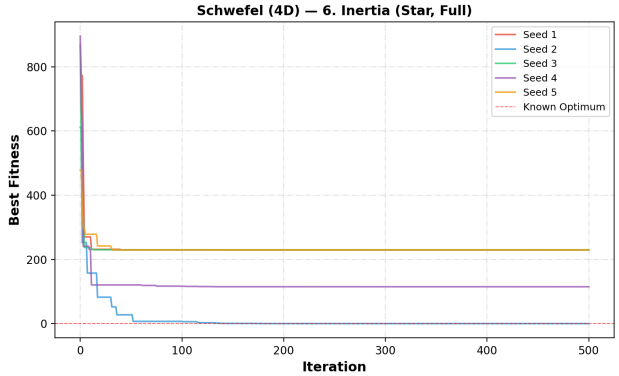

Figure 22 Convergence curves for inertia (Star, Full) on the Schwefel function (4D)

5.7. Constriction (Star, Full)

The constriction-based PSO shows moderate performance on the 4D Schwefel function but does not achieve strong results. As shown in Table 18, the algorithm does not reach the global optimum in any run, and the mean fitness remains relatively high with noticeable variability. Although the constriction coefficient is designed to improve convergence stability, it appears to limit exploration in this problem. From Figure 23, the convergence curves show that the algorithm stabilizes quickly but often at suboptimal solutions. Overall, constriction alone does not provide sufficient improvement and struggles with the complex landscape of the Schwefel function.

Table 18 Schwefel (4D) PSO results for constriction (Star, Full)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	114.7534	420.9687, 420.9687, -512.0000, 420.9687		0.326
2	114.7534	420.9687, -512.0000, 420.9687, 420.9687		0.328
3	229.5068	420.9687, 420.9687, -512.0000, -512.0000		0.326
4	233.1917	-512.0000, 420.9687, -302.5249, 420.9687		0.329
5	229.5068	-512.0000, 420.9687, -512.0000, 420.9687		0.326
Mean	**184.3424**	**Std: 56.8351**	**Best: 114.7534**	**Worst: 233.1917**

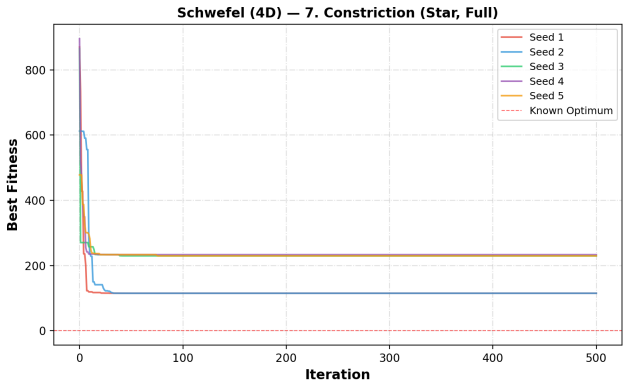

Figure 23 Convergence curves for constriction (Star, Full) on the Schwefel function (4D)

5.8. Inertia + Constriction

The combination of inertia and constriction shows mixed performance on the 4D Schwefel function. As seen in Table 19, one run successfully reaches the global optimum (fitness ≈ 0), but the other runs remain far from optimal, resulting in a high mean fitness and large standard deviation. This indicates that the method is not consistently reliable. From Figure 24, the convergence curves show that while one particle is able to continue improving and reach the optimal solution, the others quickly stabilize at suboptimal values. Overall, combining inertia and constriction provides some benefit, but it does not consistently outperform simpler approaches and still struggles with the complexity of the problem.

Table 19 Schwefel PSO results for inertia + constriction

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.5068	420.9687, 420.9687, -512.0000, -512.0000		0.325
2	-0.0000	420.9687, 420.9687, 420.9687, 420.9687		0.327
3	233.1917	420.9687, 420.9687, -512.0000, -302.5249		0.328
4	233.1917	-512.0000, 420.9687, -302.5249, 420.9687		0.326
5	229.5068	-512.0000, 420.9687, -512.0000, 420.9687		0.327
Mean	**185.0794**	**Std: 92.5544**	**Best: -0.0000**	**Worst: 233.1917**

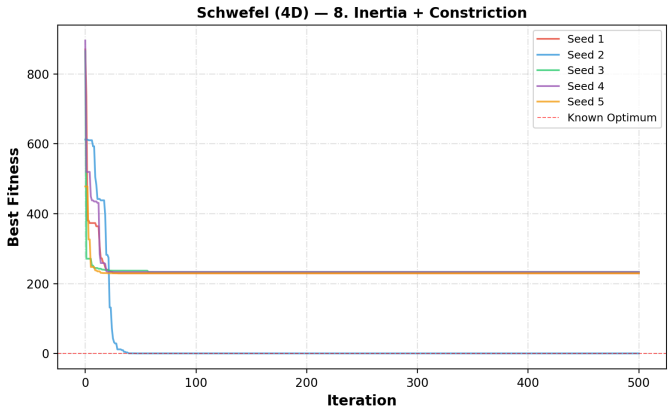

Figure 24 Convergence curves for inertia + constriction on the Schwefel function

5.9. More Particles (Inertia+Constr)

Increasing the number of particles to 60 significantly improves performance on the 4D Schwefel function. As shown in Table 20, multiple runs successfully reach the global optimum (fitness ≈ 0), demonstrating better exploration of the search space. Although some runs still get trapped in local minima, the overall mean fitness is much lower compared to previous configurations. From Figure 25, the convergence curves show that several particles are able to escape poor regions and continue improving toward the optimal solution. However, this improvement comes with a higher computational cost, as reflected in the increased runtime. Overall, using more particles enhances reliability and increases the chances of finding the global optimum.

Table 20 Schwefel PSO results for more particles (Inertia+Constr)

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)	Runtime (s)
1	-0.0000	420.9687, 420.9687, 420.9687, 420.9687	0.835
2	-0.0000	420.9687, 420.9687, 420.9687, 420.9687	0.834
3	229.5068	420.9687, -512.0000, -512.0000, 420.9687	0.837

4	-0.0000	420.9687, 420.9687, 420.9687, 420.9687		0.831
5	229.5068	-512.0000, 420.9687, -512.0000, 420.9687		0.835
Mean	**91.8027**	**Std: 112.4349**	**Best: -0.0000**	**Worst: 229.5068**

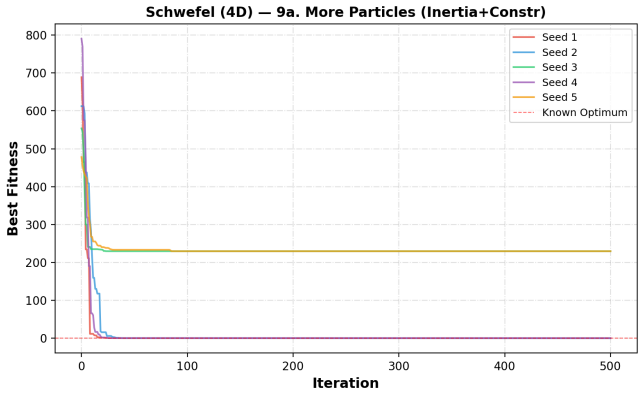

Figure 25 Convergence curves for more particles (Inertia+Constr) on the Schwefel function

5.10. Fewer Particles

Reducing the number of particles to 15 leads to poor performance on the 4D Schwefel function. As shown in Table 21, none of the runs reach the global optimum, and all solutions remain far from the optimal value, resulting in a high mean fitness. Although the standard deviation is moderate, all runs consistently perform poorly. From Figure 26, the convergence curves show that the algorithm quickly stabilizes but in suboptimal regions, with very limited improvement after the early iterations. While the runtime is lower due to fewer particles, the lack of exploration significantly reduces solution quality. Overall, using fewer particles is not effective for this complex high-dimensional problem.

Table 21 Schwefel PSO results fewer particles

Seed	Best Fitness	Best Position (x_1, x_2, x_3, x_4)		Runtime (s)
1	229.5068	-512.0000, 420.9687, -512.0000, 420.9687		0.143
2	344.2602	-512.0000, 420.9687, -512.0000, -512.0000		0.141
3	233.1917	420.9687, 420.9687, -512.0000, -302.5249		0.141
4	229.5068	420.9687, -512.0000, 420.9687, -512.0000		0.141
5	233.1917	-512.0000, 420.9687, -302.5249, 420.9687		0.142
Mean	**253.9315**	**Std: 45.1944**	**Best: 229.5068**	**Worst: 344.2602**

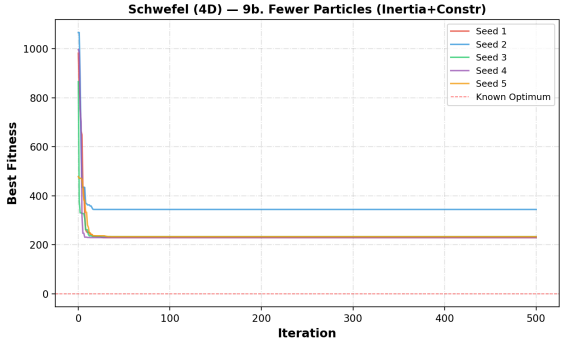

Figure 26 Convergence curves for fewer particles on the Schwefel function (4D)

5.11. Overall Comparison

The overall comparison of all PSO variants on the 4D Schwefel function is summarized in Table 22, which reports the mean, standard deviation, best, and worst fitness values for each method. As required in the assignment, each variant was evaluated across five seeds to assess both performance and consistency. From the table, the ring topology (full model**)** achieves the best overall mean fitness, indicating strong performance on this challenging problem. The cognition-only model and more particles configuration also show relatively good results. In contrast, the full model, selfless, and especially the fewer particles configuration perform poorly, with high mean fitness values and difficulty escaping local minima.

The differences between methods are further illustrated in Figure 27 (boxplot) and Figure 28 (mean ± std plot**)**. These figures show that the ring topology and cognition-only models generally achieve lower fitness values with more consistent performance, while other models exhibit larger variability. The inertia-based methods are able to reach the global optimum in some runs (fitness ≈ 0), but their performance is inconsistent, as reflected by large standard deviations. The constriction and inertia+constriction models do not show clear improvement, often stabilizing at suboptimal values. Overall, these figures highlight that maintaining diversity and gradual convergence is more effective than rapid convergence.

Finally, the effect of swarm size is clearly shown in Figure 29, which compares different particle counts. Increasing the number of particles improves the algorithm's ability to explore the search space and find the global optimum in some runs, as also seen in Table 22. However, this comes with increased computational cost. On the other hand, reducing the number of particles leads to very poor performance, with all runs getting stuck far from the optimum. Overall, the results confirm that the Schwefel function is much more difficult than the Eggholder function, and that successful optimization depends on maintaining diversity and sufficient swarm size.

Table 22 Summary of all PSO variants for the Schwefel function

Variant	Particles	Mean	Std	Best	Worst
Full Model (Star)	50	207.4835	44.5232	118.4373	229.8916
Cognition Only (φ_2=0)	50	93.5484	34.3284	36.6424	130.7328
Social Only (φ_1=0)	50	185.9711	87.2408	11.4895	229.7059
Selfless (φ_1=0, g≠i)	50	206.6517	45.8532	114.9455	229.7033
Ring Topology (Full)	50	79.3163	27.0986	41.3782	122.0768
Inertia (Star, Full)	50	160.6548	91.8027	-0.0000	229.5068
Constriction (Star, Full)	50	184.3424	56.8351	114.7534	233.1917
Inertia + Constriction	50	185.0794	92.5544	-0.0000	233.1917
More Particles (Inertia+Constr)	100	91.8027	112.4349	-0.0000	229.5068
Fewer Particles (Inertia+Constr)	25	253.9315	45.1944	229.5068	344.2602

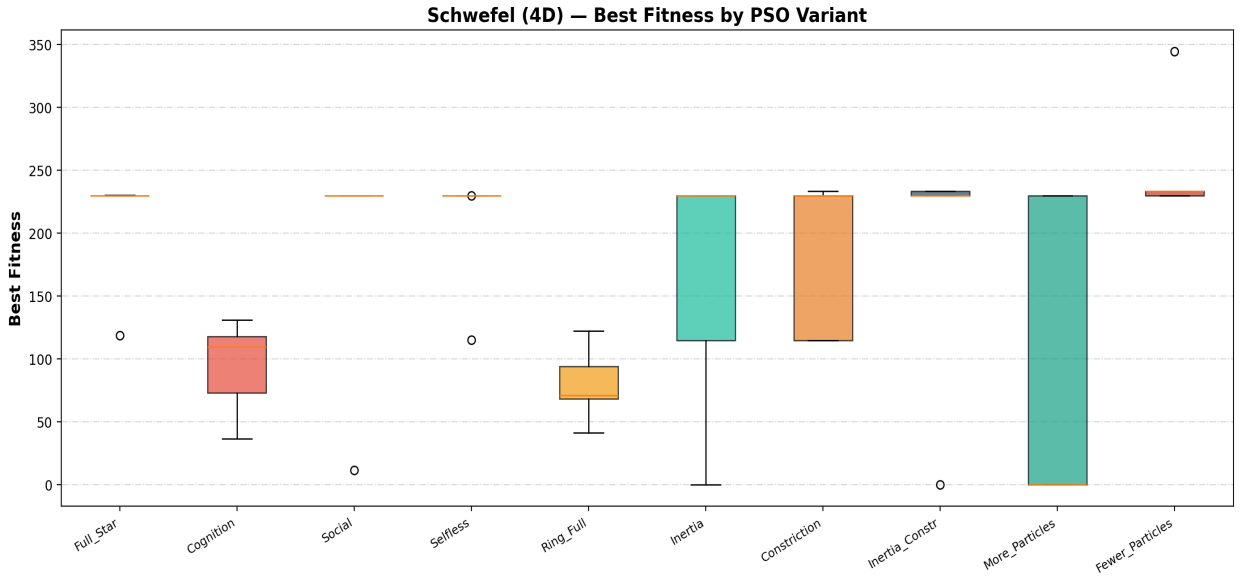

Figure 27 Boxplot comparison of all PSO variants on the Schwefel function

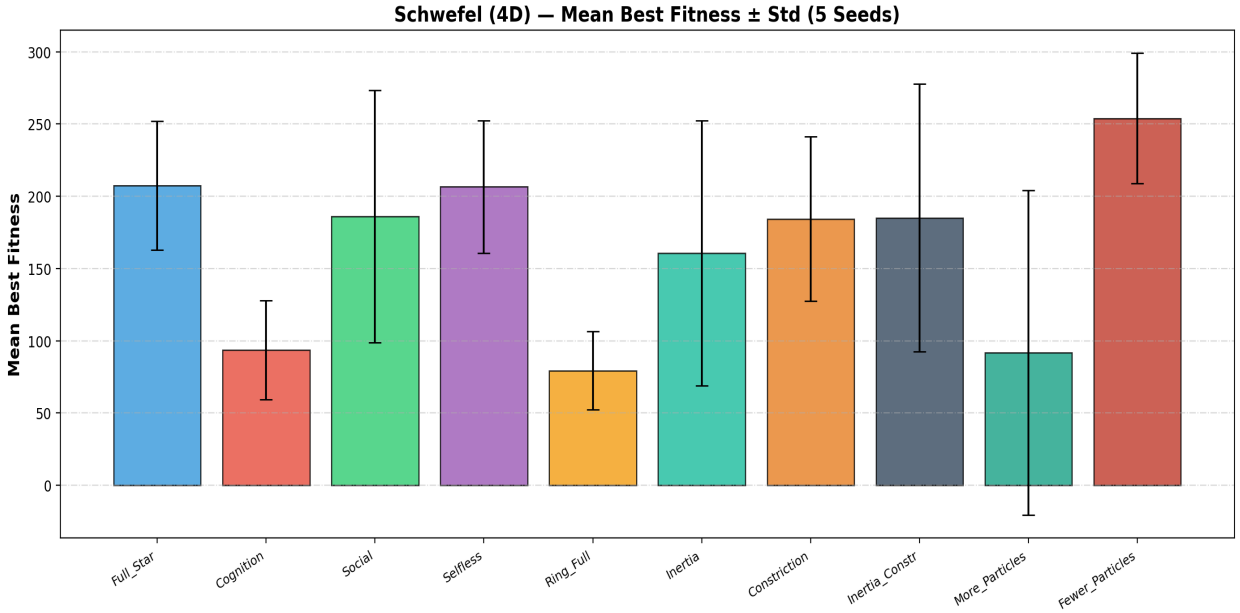

Figure 28 Mean best fitness (± std) for all PSO variants on the Schwefel function

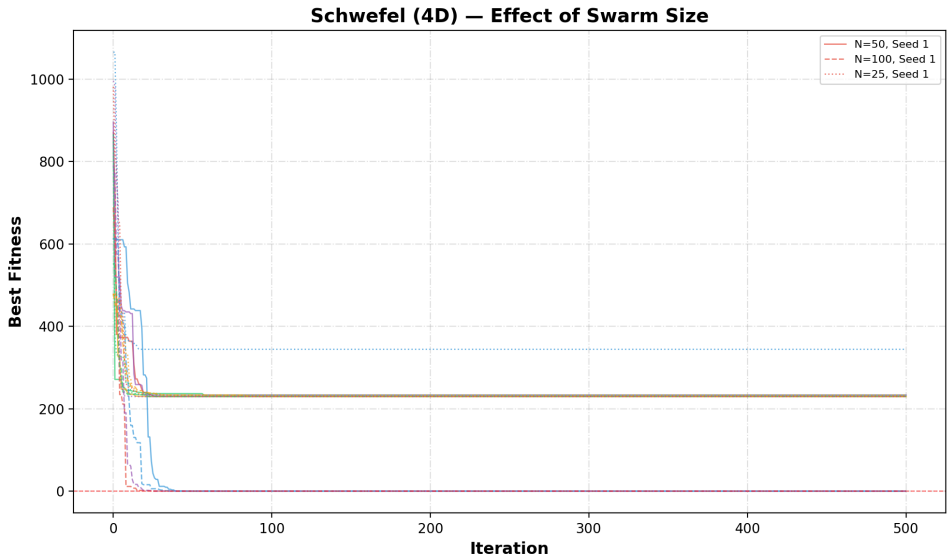

Figure 29 Effect of swarm size on PSO performance for the Schwefel function

6. Conclusion

This project showes that Particle Swarm Optimization is an effective method for solving continuous optimization problems, particularly for complex multimodal functions. For the Eggholder function, several configurations were able to consistently achieve near-optimal solutions, while for the more challenging 4D Schwefel function, some variants successfully reached the global optimum. Overall, the results confirm that PSO can provide strong performance when properly configured, but its effectiveness depends heavily on the choice of parameters and model design.

Different PSO variants exhibit distinct search behaviors. Methods that maintain diversity, such as the cognition-only model and ring topology, consistently perform better by avoiding premature convergence. In contrast, the social-only and selfless models tend to converge quickly but often get trapped in local minima due to insufficient exploration. The inertia weight plays a key role in improving performance by allowing a smooth transition from global exploration to local exploitation. On the other hand, the constriction coefficient provides theoretical stability but does not consistently outperform inertia in practice, and combining both methods yields only limited improvements.

Swarm size is another critical factor affecting performance. Larger swarms improve the coverage of the search space and increase the likelihood of finding the global optimum, especially in higher-dimensional problems like the Schwefel function, although this comes at the cost of increased runtime. Smaller swarms significantly reduce performance due to limited exploration. Additionally, techniques such as velocity restriction and proper boundary handling are essential for stable behavior. Overall, the results highlight that maintaining diversity, using appropriate parameter settings, and selecting suitable swarm sizes are key to achieving reliable and high-quality solutions with PSO.

5. Python code

The Python codes used for this project are shared in GoogleColab, which is an online, free-access, and reproducible platform, as follows:

Python code

Also, I created a GitHub repository to publish this report and codes publicly available for anyone interested to use.

GitHub

AI Statement

Grammarly and ChatGPT were used only for grammar correction and to improve the clarity of the text I wrote, while I personally conducted all analysis, coding, and results.

References

1. Alice E. Smith (2026). Presentation for the Intelligence Optimization course for Spring semester. University of Alabama.
2. Kennedy, J., & Eberhart, R. (1995, November). Particle swarm optimization. In Proceedings of ICNN'95-international conference on neural networks (Vol. 4, pp. 1942-1948). ieee.
3. Wang, D., Tan, D., & Liu, L. (2018). Particle swarm optimization algorithm: an overview. Soft computing, 22(2), 387-408.